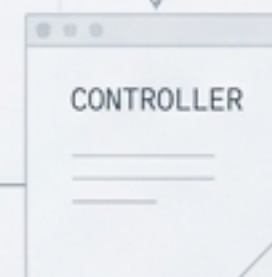
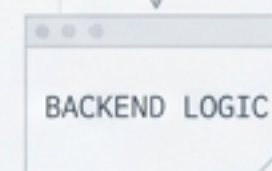
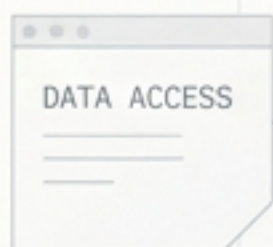




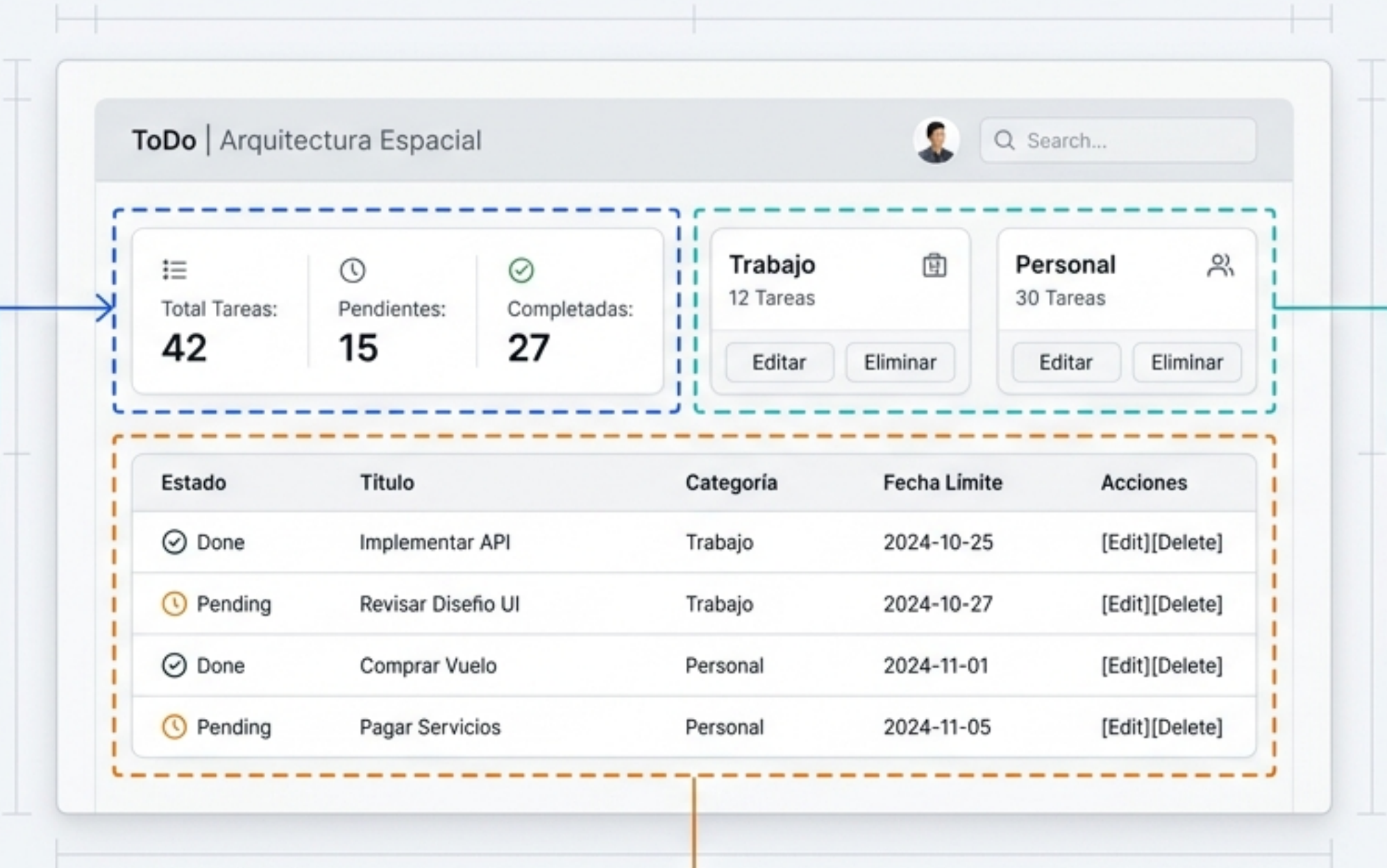
Anatomía de una App de Tareas: De C# a la Web

Una deconstrucción visual de **ASP.NET Core**
Razor Pages para desarrolladores.



Resumen Global
(IndexModel)

Muestra métricas de tareas y categorías.



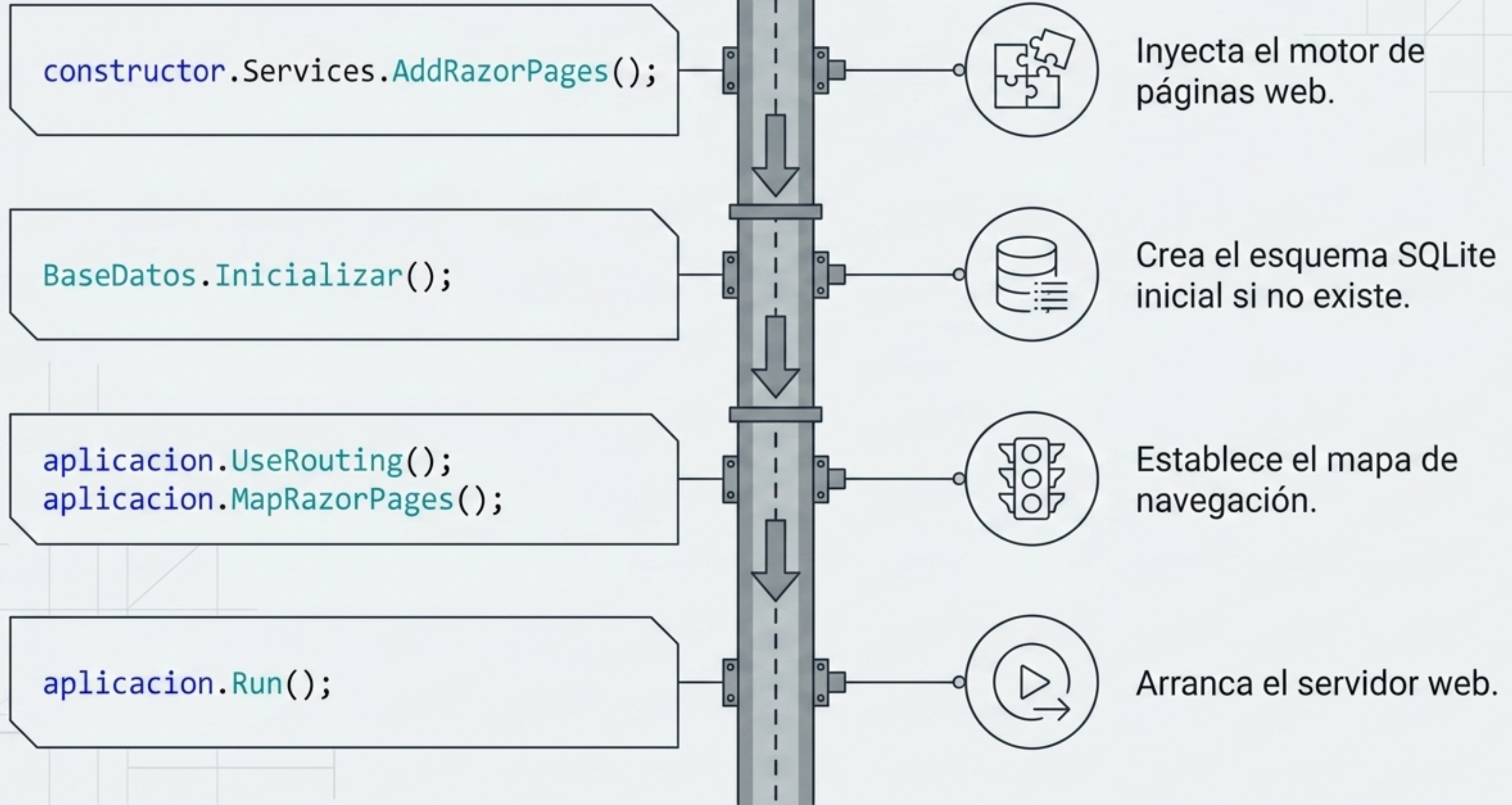
Gestión de Entidades
(CategoriasIndexModel)

Tarjetas iterables con acciones CRUD.

Tabla de Datos (TareasIndexModel)

Lista dinámica con indicadores de estado visuales.

El Motor de Arranque (Program.cs)



Clase Categoria (Independiente)

```
public class Categoria {  
    public int Id { get; private set; }  
    public string Nombre { get; set; }  
    public string Descripcion { get; set; }  
}
```

Clase Tarea (Dependiente)

```
public class Tarea {  
    public int Id { get; private set; }  
    public string Titulo { get; set; }  
    public bool Completada { get; set; }  
    public Categoria Categoria { get; set; }  
}
```

Entidades en C# puras (POCOs).
Sin lógica de interfaz gráfica, listas
para ser mapeadas.


```
using Microsoft.Data.Sqlite;
```

```
public int Id  
public string Titulo  
public bool Completada  
public int CategoriaId
```



SQL Engine

Persistencia Relacional

Cada instancia del objeto en memoria se traduce en una fila física en el archivo local de la base de datos.

Id	Titulo	Completada	CategoriaId
1	'Comprar leche'	false	2

El Cerebro: PageModel

Index.cshtml.cs

```
public class IndexModel  
: PageModel  
{  
    ...  
}
```

Ejecuta la lógica del servidor, consulta la base de datos y prepara las variables.



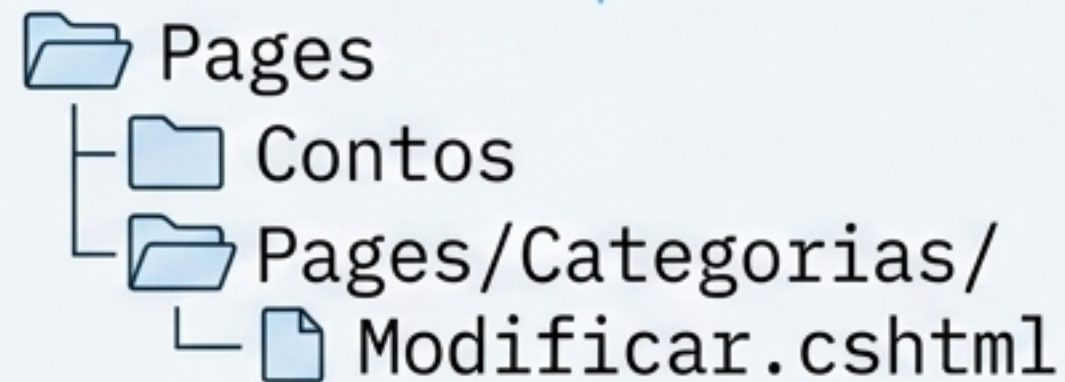
El Rostro: Vista Razor

Index.cshtml

```
@page  
@model IndexModel
```

Renderiza la interfaz utilizando las propiedades expuestas por el modelo.

localhost:5000/Categorias/Modificar/5



@page "{id:int}"

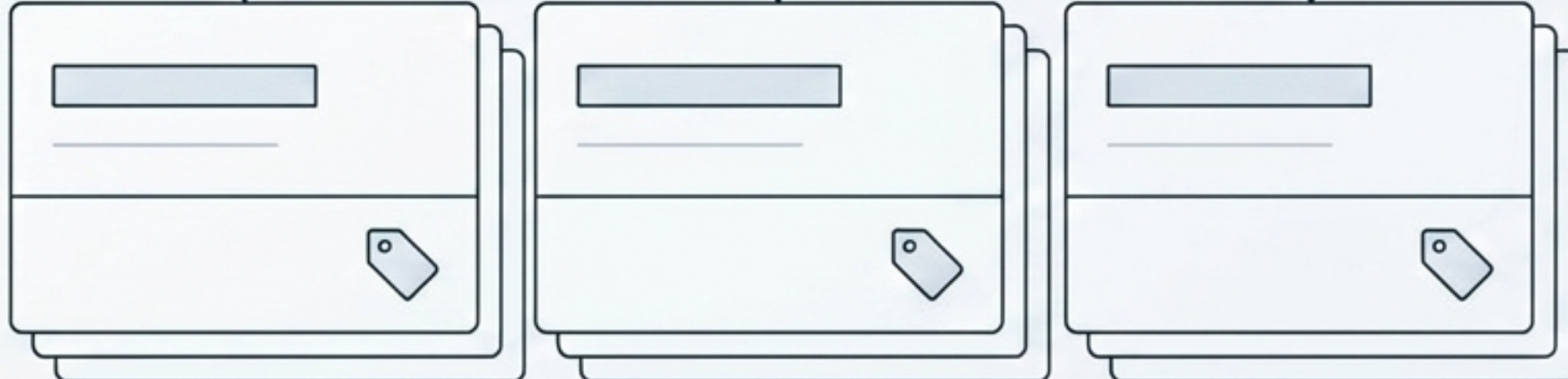
Enrutamiento Inteligente

La directiva `@page "{id:int}"` captura el '5' de la URL y lo convierte en un parámetro de tipo entero, bloqueando accesos no válidos automáticamente.


```
public class CategoriasIndexModel : PageModel
{
    public List<Categoria> Categorias { get; set; }
}
```

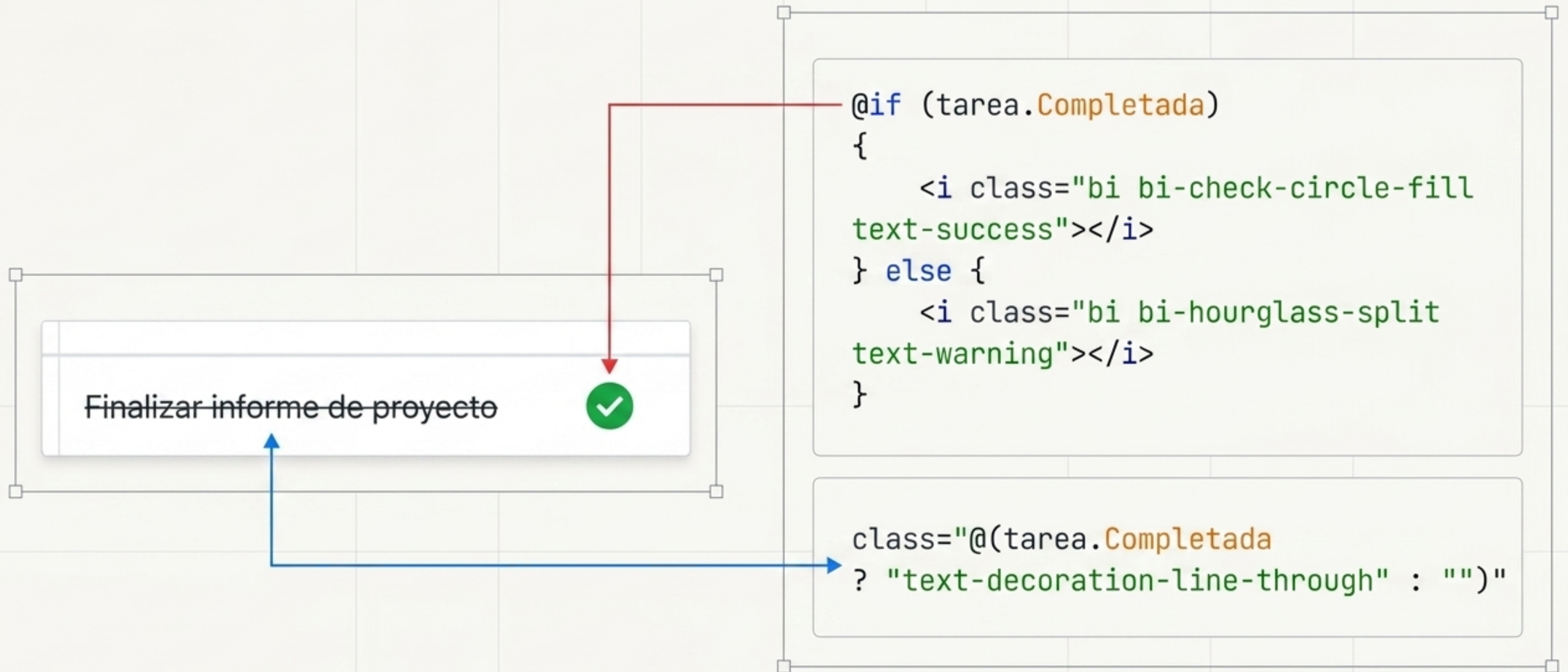


```
@foreach (Categoria categoria in Model.Categorias)
```



De C# al DOM

El modelo expone una lista pública; la vista itera sobre ella renderizando HTML puro por cada elemento.

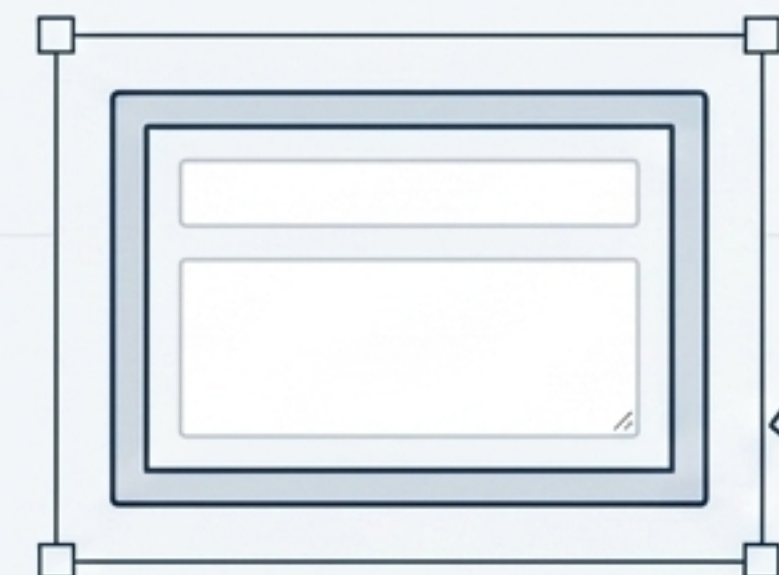


Simbiosis Perfecta. Transiciones fluidas entre HTML estático y condicionales lógicas de C#.

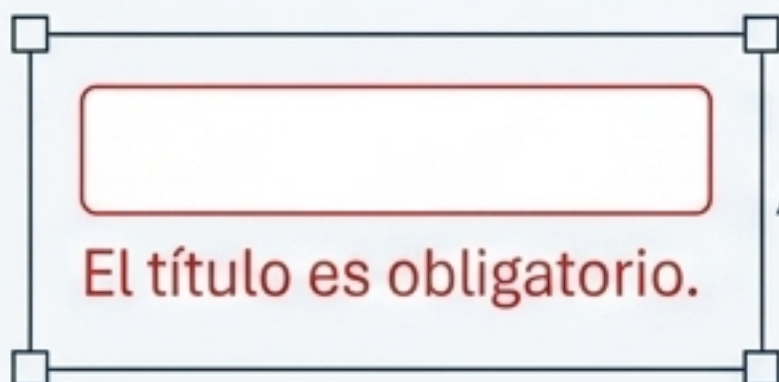


Enlace de Datos Bidireccional

El atributo `[BindProperty]` captura automáticamente los datos del formulario (POST) y los inyecta en la propiedad de C# sin necesidad de mapeo manual.



```
[Required(ErrorMessage = "El título es obligatorio.")]  
public string Titulo { get; set; }
```



Validación Elegante

Protege el servidor y alerta al usuario simultáneamente con una sola línea de código declarativo.

The Trigger

Formulario HTML

```
<form method="post" asp-page-handler="Eliminar">  
  <input type="hidden" name="id" value="@tarea.Id" />  
  <button type="submit">Eliminar</button>  
</form>
```

The Handler

Código C#

```
public IActionResult OnPostEliminar(int id) { ... }
```

Handlers Específicos

En lugar de controladores masivos, las Razor Pages dirigen acciones concretas a métodos dedicados (OnPost[Handler]), pasando los IDs de forma segura y oculta.

Fase 1: Acción C#

```
TempData["Mensaje"] = "Tarea guardada";  
return RedirectToPage("./Index");
```

Fase 2: Redirección



Mensajes Efímeros.
TempData sobrevive exactamente
una redirección. Perfecto para
notificaciones de éxito o alertas
temporales.

Fase 4: Limpieza



Fase 3: Renderizado Razor

✓ Tarea guardada con éxito ✕

```
@if (TempData["Mensaje"] != null)
```



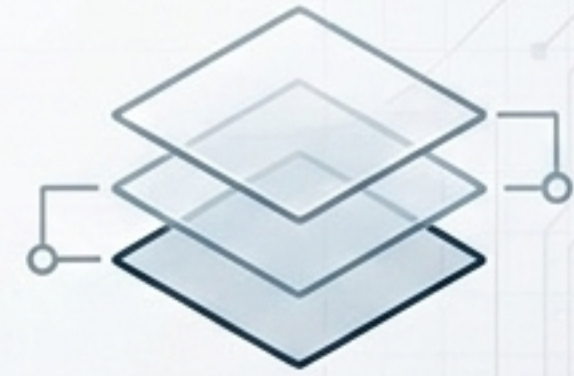

Menos Código *Boilerplate.*

La arquitectura **PageModel1** unifica **vista** y controlador, eliminando abstracciones innecesarias.



Tipado Fuerte de Extremo a Extremo.

Desde la base de datos hasta la plantilla **HTML**, el compilador de **C#** detecta errores antes de la ejecución.



Separación de Intereses Clara.

El diseño **HTML** pertenece a la **Vista**, la lógica de negocio al **Modelo**, manteniendo un proyecto limpio y escalable.

ASP.NET Core Razor Pages: Productividad, rendimiento y estructura para la web moderna.